

LAB 5

Constructor Overloading in C++

Default, Partial, and Fully Parameterized Constructors

Learning Objectives

By the end of this lab, you should be able to:

- Explain what a constructor is and what constructor overloading means.
- Recognize that overloaded constructors must differ in number, type, or order of parameters.
- Write a default constructor that initializes an object with sensible default values.
- Write parameterized constructors (partial and full) to initialize objects at creation time.
- Add validation logic inside a constructor to reject invalid input.
- Understand how the compiler picks the correct constructor based on the arguments you pass.
- Know what a destructor is and when it runs.

1. Why Do We Need Constructor Overloading?

In real programs, objects are not always created with the same information. Consider a Student class:

- Sometimes you have **no information yet** — you just want an empty Student object.
- Sometimes you only know the **name**.
- Sometimes you have **everything** — name, age, and CGPA.

If you only had one constructor, you would have to fill in dummy values every time, or write separate setData(), setName(), setAll() functions. Constructor overloading is a cleaner solution: the class itself offers several ways to be constructed.

The problem with a single constructor

If a class only has one constructor that takes (name, age, CGPA), then every single object must supply all three values at construction time — even when you don't know them yet. This forces ugly placeholder values like "", 0, 0.0 everywhere in your code.

2. What Is a Constructor? (Quick Refresher)

A constructor is a special member function that runs automatically when an object is created. It has three defining properties:

- **Same name as the class.**
- **No return type** — not even void.
- **Called automatically** — you never call it explicitly with a function-call syntax.

 Remember

A constructor sets up an object. **Overloading** just means writing **several different constructors** for the same class, each accepting a different set of parameters.

3. How Constructor Overloading Works

A class is allowed to have many constructors as long as each one has a different parameter signature. The compiler looks at the arguments you pass when creating an object and automatically picks the constructor that matches.

Rules for constructor overloading

- All constructors share the same name (the class name).
- They must differ in the number of parameters, OR
- the type of parameters, OR
- the order of parameters.

 Quick mental model

Think of overloaded constructors as multiple "doors" into the same class. Each door accepts a different set of keys (parameters). The compiler decides which door you walked through by looking at the keys in your hand.

4. Code 1 — The Default Constructor

Goal: a constructor that takes no arguments and fills the object with safe default values.

The Code

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int    age;
    float  cgpa;
public:
    Student();           // Default constructor (no parameters)
    void display();
};

// Default Constructor
Student::Student() {
    name = "Unknown";
    age  = 0;
    cgpa = 0.0;
}

void Student::display() {
    cout << "Name: " << name
         << ", Age: " << age
         << ", CGPA: " << cgpa << endl;
}

int main() {
    Student s1;           // No arguments → default constructor runs
    s1.display();
}
```

Code Breakdown

Code	What it does
<code>Student();</code>	← Declares a constructor with no parameters — this is the default constructor.
<code>Student::Student() {</code>	← Definition of the default constructor outside the class.
<code>name = "Unknown";</code>	← Give name a safe placeholder value.
<code>age = 0;</code>	← Initialize age to 0 so it is never garbage.
<code>cgpa = 0.0;</code>	← Same idea for cgpa.

Code	What it does
<code>Student s1;</code>	← Notice: no parentheses, no arguments. This triggers the default constructor.
<code>s1.display();</code>	← Prints the default values to confirm the constructor ran.

Expected Output

► Output

Name: Unknown, Age: 0, CGPA: 0



Why have a default constructor at all?

Without one, you cannot write `Student s1;` — the compiler will complain. Default constructors are essential when you create arrays of objects or when an object is created before you know its real values.

5. Code 2 — Partial Parameterized Constructor

Goal: a second constructor that takes one piece of information — the name — and uses defaults for the rest. This is also our first taste of validation inside a constructor.

The Code

```
// Partial Parameterized Constructor (with validation)
Student::Student(string n) {
    if (n.empty()) {
        cout << "Invalid name! Setting default name" << endl;
        name = "Unknown";
    } else {
        name = n;
    }
    age = 0;
    cgpa = 0.0;
}

int main() {
    Student s2("Karim");    // One argument → this constructor runs
    s2.display();
}
```

Code Breakdown

Code	What it does
<code>Student::Student(string n)</code>	← A second constructor with the same name but a different signature — overloading.

Code	What it does
<code>if (n.empty())</code>	← VALIDATION: refuse an empty string as a name.
<code>name = "Unknown";</code>	← Fall back to a safe default if validation failed.
<code>else name = n;</code>	← Otherwise, store the provided value.
<code>age = 0; cgpa = 0.0;</code>	← Set the remaining fields to defaults — the caller didn't supply them.
<code>Student s2("Karim");</code>	← One string argument, so the compiler picks this constructor.

Expected Output

► Output

Name: Karim, Age: 0, CGPA: 0

⚠ What if you wrote `Student s2("");`?

The empty-string check would trigger, print "Invalid name! Setting default name", and name would be saved as "Unknown" instead. The object is never left in a broken state — that is the whole point of validating inside the constructor.

6. Code 3 — Fully Parameterized Constructor with Full Validation

Goal: a third constructor that accepts all three fields at once and validates each one independently.

The Code

```
Student::Student(string n, int a, float c) {
    // Name validation
    if (n.empty()) {
        cout << "Invalid name! Setting default name.\n";
        name = "Unknown";
    } else {
        name = n;
    }

    // Age validation
    if (a < 0) {
        cout << "Invalid age! Setting age = 0.\n";
        age = 0;
    } else {
        age = a;
    }

    // CGPA validation
    if (c < 0.0 || c > 4.0) {
```

```

        cout << "Invalid CGPA! Setting CGPA = 0.0.\n";
        cgpa = 0.0;
    } else {
        cgpa = c;
    }
}

int main() {
    Student s3("Rahim", 25, 3.5);    // Three arguments → this constructor runs
    s3.display();
}

```

Code Breakdown

Code	What it does
Student(string n, int a, float c)	← Three-parameter constructor — completely different signature from the other two.
if (n.empty())	← Name check, same as before.
if (a < 0)	← Age cannot be negative — a person cannot be -5 years old.
if (c < 0.0 c > 4.0)	← CGPA must sit on a 0.0–4.0 scale; reject anything outside it.
name = n; age = a; cgpa = c;	← Accept the values once they pass all the checks.
Student s3("Rahim", 25, 3.5);	← Three arguments are provided, so the three-parameter constructor is chosen automatically.

Expected Output

► Output

Name: Rahim, Age: 25, CGPA: 3.5

📦 Each field has its own validation

Notice we did NOT combine the checks into one big if. Each field is validated independently, and each invalid field falls back to its own safe default. This way, if only the CGPA is wrong, the name and age you provided are still kept.

7. Code 4 — All Three Constructors Together (with Destructor)

Now we put all three constructors into a single class. We also add a destructor — a special function that runs automatically when an object is destroyed (for example, when it goes out of scope at the end of main).

The Complete Program

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int age;
    float cgpa;
public:
    Student(); // Default constructor
    Student(string n); // Partial parameterized
    Student(string n, int a, float c); // Fully parameterized
    void display();
    ~Student(); // Destructor
};

// Default Constructor
Student::Student() {
    name = "Unknown";
    age = 0;
    cgpa = 0.0;
}

// Partial Parameterized Constructor (with validation)
Student::Student(string n) {
    if (n.empty()) {
        cout << "Invalid name! Setting default name" << endl;
        name = "Unknown";
    } else {
        name = n;
    }
    age = 0;
    cgpa = 0.0;
}

// Fully Parameterized Constructor (with full validation)
Student::Student(string n, int a, float c) {
    if (n.empty()) { cout << "Invalid name! Setting default name.\n"; name = "Unknown"; }
    else { name = n; }

    if (a < 0) { cout << "Invalid age! Setting age = 0.\n"; age = 0; }
    else { age = a; }

    if (c < 0.0 || c > 4.0) { cout << "Invalid CGPA! Setting CGPA = 0.0.\n"; cgpa = 0.0; }
```

```

        else                { cgpa = c; }
    }

    // Display Function
    void Student::display() {
        cout << "Name: " << name
              << ", Age: " << age
              << ", CGPA: " << cgpa << endl;
    }

    // Destructor
    Student::~~Student() {
        cout << "Destructor Called for " << name << endl;
    }

    int main() {
        Student s1;                // → Default constructor
        Student s2("Karim");       // → Partial parameterized
        Student s3("Rahim", 25, 3.5); // → Fully parameterized

        cout << endl << endl;
        s1.display();
        s2.display();
        s3.display();

        cout << endl << "End of main function" << endl << endl;
        return 0;
    }

```

Which Constructor Runs for Which Object?

Code	What it does
Student s1;	← No arguments → Default Constructor runs.
Student s2("Karim");	← One string argument → Partial Parameterized Constructor runs.
Student s3("Rahim", 25, 3.5);	← Three arguments (string, int, float) → Fully Parameterized Constructor runs.
~Student()	← Destructor — runs automatically for s1, s2, s3 when main ends, in reverse order of creation.

Expected Output

► Output

```

Name: Unknown, Age: 0, CGPA: 0
Name: Karim, Age: 0, CGPA: 0

```


Name: Rahim, Age: 25, CGPA: 3.5

End of main function

Destructor Called for Rahim

Destructor Called for Karim

Destructor Called for Unknown



Why the destructors print in reverse order

Local objects are destroyed in the reverse of the order they were created (last-in, first-out). s3 was created last, so its destructor runs first. This is C++'s built-in cleanup mechanism — and it is deterministic, which is one of C++'s superpowers.



Key idea — compiler picks the constructor automatically

You do not have to tell the compiler which constructor to call. It looks at the arguments you supplied (count, type, and order) and matches them to the constructor with a compatible signature. This is the whole magic of overloading.

8. Summary

Four ideas to take away from this lab:

- **Constructors initialize objects** automatically when they are created — they share the class's name and have no return type.
- **Overloading means many constructors.** Each must differ in the number, type, or order of parameters.
- **Validate inside the constructor.** An object should never come into existence in an invalid state.
- **Destructors clean up.** They run automatically when an object's lifetime ends, in reverse order of creation.

9. Practice Tasks

Task 1 — Book Class with Overloaded Constructors

Create a class `Book` with the following private data members:

- `title` (string)
- `author` (string)
- `price` (double)

Required Constructors (constructor overloading)

- **Default constructor** — sets `title` = "Unknown", `author` = "Unknown", `price` = 0.
- **Constructor with title only** — `author` and `price` stay at defaults.
- **Constructor with title and author** — `price` stays at default.
- **Constructor with title, author, and price** — all three fields supplied.

Required Member Function

- **`display()`** — prints `title`, `author`, and `price` in a neat format.

In `main()`

Create at least 3 `Book` objects, each using a different constructor, and call `display()` on each one to show the output.

Hints for Task 1

Don't forget `#include <string>` and `using namespace std;`

Test the price-validating case too: try giving a negative price and refuse it in the setter or constructor (bonus).

If your default constructor and your one-argument constructor both look similar, that's fine — overloading is about the signature, not the body.

Optional challenge

Add a destructor to `Book` that prints a short message when the object is destroyed. Run your program and observe the order in which the destructors fire. Does it match what you saw in Code 4?

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST